
Advanced Designer

Welcome

Meet the trainer:

Joe Fagan
Knowledge Graph Consultant
joe.fagan@stardog.com
+44 7788 148772



Course Objectives

After completing this training, students will be able to:

- Understand Triples, IRIs and Namespaces
- Perform advanced data modeling and complex mapping
- Read and interpret exported project files
- Configure and map data using Virtual Graphs
- Design and publish production-ready data models
- Create and manage Sensitive Properties



Course Overview

Description:

This Advanced Designer training is intended for students who have completed the *Meet Stardog*, *Explorer*, and *Designer* trainings. Participants will deepen their knowledge of modeling and mapping using Stardog Designer, learning how to create and customize IRIs and namespaces, understand schemas and exported schema files, perform advanced mappings, configure sensitive properties, and explore virtual graph configuration.

By the end of the training, students will be able to design and publish a production-ready data model, and will be prepared to take the Designer certification exam.

Audience:

This training is designed for users of the Stardog Platform who are responsible for designing and developing knowledge graphs.

Prerequisites:

A Stardog Cloud account and endpoint to connect to
Completion of the *Meet Stardog*, *Explorer* and *Designer Training*



Agenda

1 Understanding Triples, IRIs & Namespaces

2 Advanced data modeling and mapping

3 Read and interpret exported files

4 Map using Virtual Graphs

5 Publishing production ready data models

6 Sensitive Properties



Understanding Triples, IRIs and Namespaces

Triples

A triple is a fundamental unit of data used to represents a fact or a statement.

This is how everything in a graph is stored, including the data, the data models, ontologies, reasoning rules and axioms.

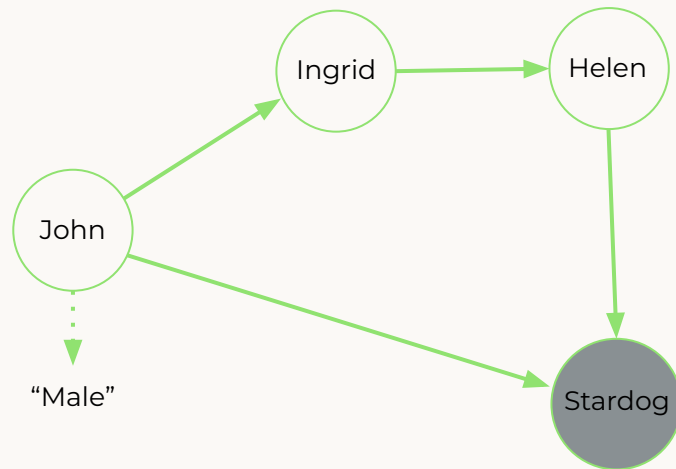
Each Triple has the form **Subject Predicate Object**

- The **Subject** is the “who” or “what”
- The **Predicate** is the property. It’s the “how” subject is connected to object. There are 2 types.
 - Attribute: eg email, phone, ssn - not shown in Explorer canvas
 - Relationship: eg livesIn, placedOrder. Links instances between classes.
- The **Object** is
 - the actual literal value (when predicate is an Attribute)
 - or another entity (when predicate is a Relationship)

If a graph is only a list of triples, where’s the graph?

Suppose our RDF graph has these triples...

Subject	Predicate	Object
Helen	WorksAt	Stardog
John	Knows	Ingrid
Ingrid	Knows	Helen
John	WorksAt	Stardog
John	Gender	“Male”

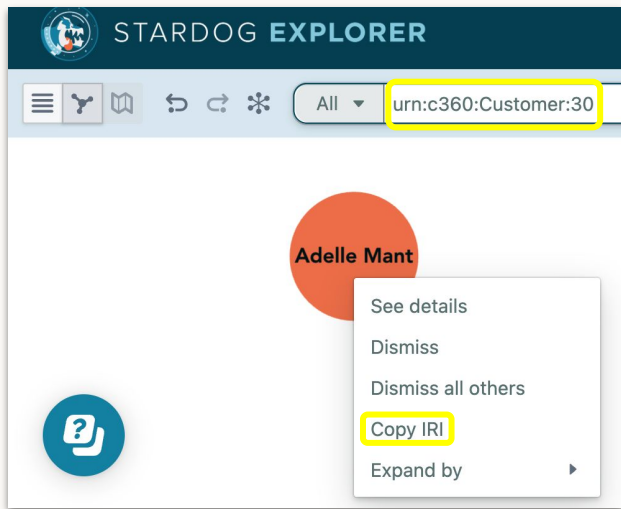


The graph emerges from re-use of nodes.



What is an IRI?

Let's begin with an example of an IRI, shown in Explorer



Here the customer with the Label "Adelle Mant" has the IRI **urn:c360:Customer:30**

This IRI uniquely and unambiguously identifies the entity with the label "Adelle Mant".

Every triple that refers to "Adelle Mant" will use this IRI

In RDF, there is a distinction between the label of a thing ("Adelle Mant"), and the thing itself, which is an IRI.

The Subject and Predicate of every triple in an RDF graph are always IRIs
The Object of every triple is either an IRI or a Literal (eg "adelle@gmail.com")

By default Designer creates the IRIs for instance data using the template
urn:{project_name}:{resource_name}:{primary_identifier}



How IRIs are used in the graph

The triple in the graph representing

“Adelle lives In Santa Monica”

Is actually...

Subject: urn:c360:Customer:30

Predicate: urn:c360:lives_in

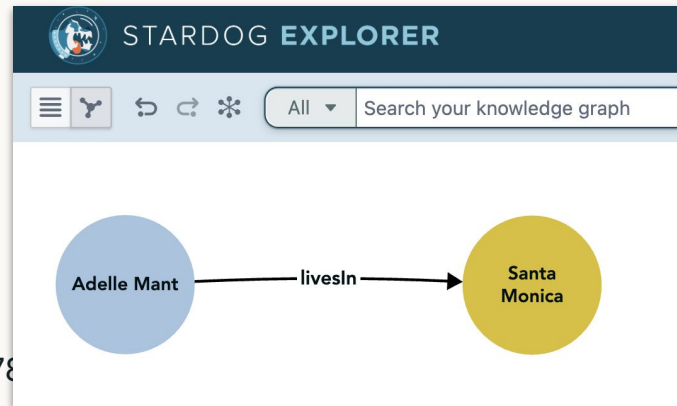
Object: urn:c360:Location:4392f3ea-c7f7-4e7a-86ec-1fb9d223ae78

By default, Designer creates IRIs for model entities using the template

`urn:{model_name}:{model_entity}`

A visualisation tool fetches and displays the **labels** of the IRIs for a human-friendly representation

Labels are for People - IRIs are for identity



Human-friendly visualisation uses Labels



The actual triple in the RDF graph uses IRIs



Internationalized Resource Identifier (IRI)

What are IRIs?

- Unique identifiers for resources: classes, instances of classes, object properties, attributes IE for everything in an RDF graph (except literals)
- Fundamental to RDF, ensuring global uniqueness.
- Used to allow RDF serve as a universal framework for data interchange on the web.

URL vs IRI:

- IRI is an extended concept of URL. URLs resolve to a webpage.
- IRI: Doesn't necessarily resolve; just a unique character sequence.
- IRIs can begin with **http:** or **https:** but also schemes like **urn:** or **tag:** or **geo:** or **isbn:**

Characteristics of IRIs:

- Often human-readable and meaningful.
 - But they are an opaque identifier
- Accompanied by a label for clarity.



Namespaces, Prefixes, and Local Names

Namespaces improve clarity and indicate where each IRI comes from. It makes the IRI more comprehensible. There are many well established namespaces of which the most common are

Prefix	Namespace	Purpose / Vocabulary	Example Terms (Prefix:LocalName)
rdf	http://www.w3.org/1999/02/22-rdf-syntax-ns#	RDF core syntax	rdf:type
rdfs	http://www.w3.org/2000/01/rdf-schema#	RDF Schema (basic data model terms)	rdfs:label, rdfs:subClassOf
owl	http://www.w3.org/2002/07/owl#	Web Ontology Language (OWL)	owl:Class
xsd	http://www.w3.org/2001/XMLSchema#	XML Schema datatypes	xsd:string, xsd:dateTime
so	http://schema.org/	Schema.org (web data vocabulary)	so:domainIncludes
geo	http://www.opengis.net/ont/geosparql#	Geo (lat/lon, position, polygons)	geo:hasGeometry

For example the full IRI of **rdf:type** is <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>

And **owl:class** has full IRI <http://www.w3.org/2002/07/owl#class>

Summary of terms.

A **Local name** is the last part of the IRI that usually comes after the last : or / or # separator

A **Namespace** is the portion up to the Local Name

A **Prefix** is a shorthand name for a namespace, to make using the IRIs less cumbersome

A **Prefixed IRI** is a shorthand name for an IRI written as **Prefix:LocalName**, eg **rdfs:label**



Namespaces, Prefixes, and Local Names



How Full IRIs are represented as Prefixed IRIs for convenience

Q. What is the Namespace of the full IRI `urn:c360:Customer:30` ?

Ans: `urn:c360:data:Customer:`

Q. What is the Local name?

Ans: `30`

Suppose we assign the prefix `Cust` to the namespace, what is the prefixed name of the IRI?

Ans: `Cust:30`



How cells in a table map to triples in the graph

How do we represent tabular data? Consider one single row from the Customer file

cid	first_name	last_name	email	ssn	phone	location	full_name
29	Setta	Wiggins	swigginss@opensource.org	614-19-5808	487-711-7474	63151510-17a0-4914-b12f-190637611700	Setta Wiggins
30	Adelle	Mant	amanтт@51.la	326-27-4675	353-449-8749	4392f3ea-c7f7-4e7a-86ec-1fb9d223ae78	Adelle Mant
31	Fancv	Espin	fespinu@pcworld.com	355-71-7352	495-389-8114	8eac8fb2-e657-44d2-a030-58fca80e4710	Fancv Espin

This single row becomes multiple triples in RDF: one triple for each column in the table

Adelle	type	Customer
Adelle	first_name	Adelle
Adelle	last_name	Mant
Adelle	email	amanтт@51.la
Adelle	ssn	326-27-4675
Adelle	phone	353-449-8749
Adelle	location	4392f3ea-c7f7-4e7a-86ec-1fb9d223ae78
Adelle	full_name	Adelle Mant
Adelle	lives_in	4392f3ea-c7f7-4e7a-86ec-1fb9d223ae78
Adelle	label	Adelle Mant

Instance Data IRIs

Model Entity IRIs

IRIs from well established namespaces

The subject of each triple is the templated IRI of the primary key

The predicate is the IRI templated from the column name (Create and Map default behaviour)

The object is the actual literal or the templated IRI of the primary key in another class



Editing IRIs and Namespaces



Why edit IRIs or Namespaces?

Designer generates sensible IRIs for new model entities and instance data

However, in the real world those defaults need to be aligned when

- integrating with an existing RDF or legacy graph
- adopting an external or industry vocabulary
- reusing data that already has established IRIs

IRIs define identity and often must align with existing data and vocabularies

Scenario

- Database already contains 'lives in' relationships using an existing IRI, eg **`https://myorg.com/vocabulary/resides`**
- Designer defaults to **`urn:c360:lives_in`**

Solution: Align the model to the existing IRI

- Required Change:
 - a. Edit IRI of lives_in relationship to **`https://myorg.com/vocabulary/resides`**
 - b. Edit the namespace to **`https://myorg.com/vocabulary/`** and the localname to **`resides`**

Result: Queries, integrations, and existing data all align without duplication.



Data Model vs Instance Data

There are two different kinds of things in the graph: **Data Model Entities** and **Instance Entities**

Data Model Entities

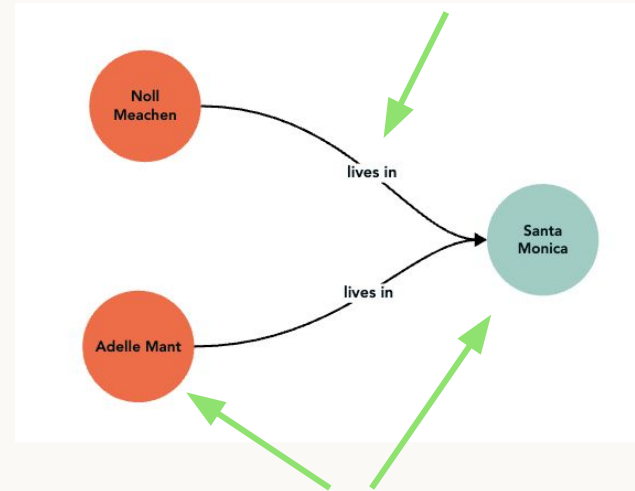
- Definitions of Classes, Attributes, Relationships, Rules
 - Eg The **Class** Customer, The **Attribute** Email, the **Relationship** lives_in
- Created in Designer
- Change infrequently

Instance Data

- Represent real records from source systems
- Examples: Adelle Mant, Santa Monica
- Each instance has its own unique IRI
- Written to the db at publish time

Data Model Entities

Created in Designer. IRIs are constructed from the Model Name and the Entity Name



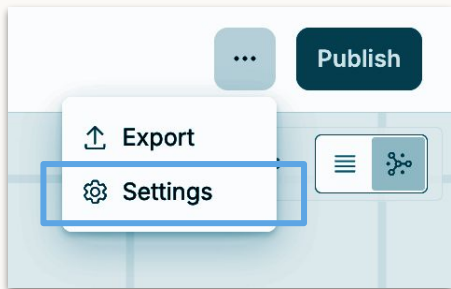
Instance Data

IRIs are constructed from the Model Name, resource name and primary identifier to ensure uniqueness. Created at Publish time.



We will see how to change the default templates so that you can control how IRIs are generated

Project Settings: modify default Namespace



To access Project Settings

From the three dots ... menu select Settings

Editing the **Instance Data Namespace** modifies the IRIs that will be generated for instance data when published.

Editing the **Data Model Namespace** modifies the template for the IRIs for new model entities.

Set Prefix will assign the prefix shown to the Data Model Namespace and is useful for anyone who will write Sparql queries on the database later. Generally leave as default.



Project Settings

Project Name: c360

Project Description:

Layout:
☐ Force-directed ☒ Horizontal Tree ☐ Vertical Tree

Instance Data Namespace: https://myorg.com/instances/

Infer class types based on shared properties ⓘ
☐ Yes ☒ No ☐ Custom per property

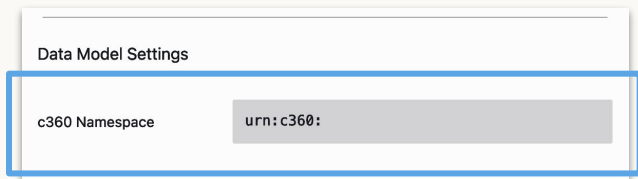
Data Model Settings

Data Model	Namespace	Set Prefix	Prefix
c360	https://myorg.com/vocabulary/	☒	voc

Data Model Namespace

Viewing and Editing an entity's IRI

Select the entity and click the `</>` icon



A screenshot of the 'Data Model Settings' dialog box. It has a title bar 'Data Model Settings'. Inside, there is a section 'c360 Namespace' with a text input field containing 'urn:c360:'. The entire dialog is highlighted with a blue border.

From Project Settings



A screenshot of the 'Class Name' dialog box. It has a title bar 'Class Name'. Inside, there is a text input field containing 'Product'. Below the input field, it says '[DATA MODEL NAMESPACE]Product'. A blue circle highlights a code icon (two arrows pointing in opposite directions) in the top right corner of the dialog.

An entity IRI is constructed or templated from

- **Data Model Namespace** (from Project Settings)
- **Localname** (the name of the entity)

Example: Product class

Shows **[DATA MODEL NAMESPACE]Product**

- The **Localname** is **Products**

From Project Settings - Data Model Settings we see

- The **Data Model Namespace** is **urn:c360:**

The full IRI is therefore **urn:c360:Products**

Note:

- Changing the Data Model Namespace affects new entities only
- Existing or imported entities are not updated

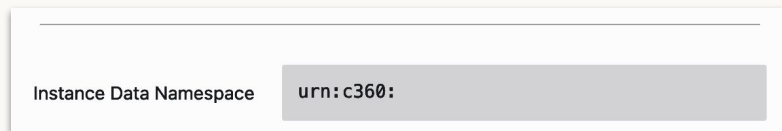


Instance Data Namespace

We saw that the instance “Adelle_Mant” has the IRI `urn:c360:Customer:30`

This IRI is constructed from

- The Instance Data Namespace: `urn:c360:` (defaults to `urn:{model}`)
- Entity name: `Customer`. (defaults to the resource name)
- The `localname`: 30. (The primary identifier of the instance. This is not editable.)

A screenshot of a web form. On the left, there is a label "Instance Data Namespace". To its right is a text input field containing the value "urn:c360:". The input field has a light gray background and a thin border.

You can view or edit the [Instance Data Namespace] in Project Settings: Click Customise advanced project details

Note: Editing the Instance Data Namespace affects only new mappings. It does not affect the mappings already in the model.

Note:

- Changing the **Instance Data Namespace** affects newly mapped and published data only
- Existing instance data is **not updated** unless it is republished

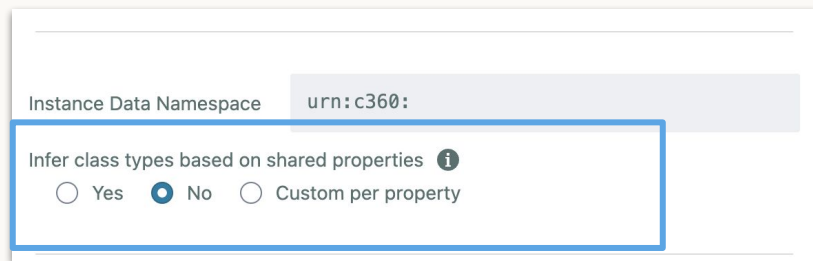


Infer Class Types Based on Shared Properties

Advanced topic. Leave this disabled unless you understand and explicitly want property-based classification

By default, this setting is turned off.

- No: (default) Stardog does not infer class types based on shared properties. This is the correct setting for most Enterprise Data
- Yes: If we define a property to be associated with a class then any instance that has the property will be inferred to be an instance of the class. **Enable only when you want property-based classification**
- Custom per property: Can then change this at the relationship and attribute level



The screenshot shows a configuration window for Stardog. At the top, there is a label 'Instance Data Namespace' followed by a text input field containing 'urn:c360:'. Below this, there is a section titled 'Infer class types based on shared properties' with an information icon (i). Under this title, there are three radio button options: 'Yes', 'No', and 'Custom per property'. The 'No' option is selected, indicated by a blue dot in the center of the radio button. The entire configuration area is enclosed in a light gray border, and the 'Infer class types based on shared properties' section is highlighted with a blue rectangular box.



Infer Class Types example

Enabled	Adding the hasWing property to the Aircraft class will create the following triple in the model. :hasWing rdfs:domain :Aircraft Then, with reasoning enabled, a compliant database (like Stardog) will try to be helpful and make anything with :hasWing an instance of :Aircraft! Example: The triples :robin_red_breast_01 a :Bird :robin_red_breast_01 :hasWing :wing10 will infer :robin_red_breast_01 a :Aircraft
Disabled (default)	the property :hasWing will be specified using schema.org as :hasWing so:domainIncludes :Aircraft This prevents a reasoner from classifying instances based on their attributes or relationships.



Designer Evolution: Standard vs Voicebox

Designer Interface and Voicebox

- Designer now supports both Standard and Voicebox modes.
- Create Project and Add Resources workflows have been refreshed
- Standard Designer, the workflow is essentially the same
 - Create a project, add resources, model and map manually.
 - Help with "create & map" functionality and "suggest mappings"
 - Enhanced "Add resources" UI
- Voicebox-enabled Designer
 - Create step includes competency questions that will inform Voicebox assisted model creation
 - Voicebox can create classes, attributes and relationships
 - Voicebox can create mapping of all resources in the model
 - Voicebox can create a simple dataset



Older Create Project

The screenshot shows the Stardog Designer web application interface. The browser address bar displays `cloud.stardog.com/u/0/designer/#/`. The application header includes the "STARDOG DESIGNER" logo, a search bar for projects, and a "New" button. A sidebar on the left lists existing projects: "Model1", "BI c360", "paws_product", "paws_main", "Data_Assets", and "person", each with its last update time. A "New Project" modal is open in the center, featuring a "Project name" field with the value "Designer Exercise". Below this, a section titled "Select how to start your project" offers four options: "Voicebox" (Build via Voicebox), "Blank Canvas" (Build from scratch), "Import a model" (Import an existing model to edit, extend, or map to), and "Connect a resource" (Leverage technical metadata from data connections to map and model). A "Start the conversation" button is visible on the right side of the interface.

cloud.stardog.com/u/0/designer/#/

STARDOG DESIGNER

Search projects

Order by Updated

+ New

Model1
Last updated: 2:54 PM Jul 10 2023

BI c360
Last updated: 3:22 PM Jul 10 2023

paws_product
Last updated: 4:37 PM Jul 14 2023

paws_main
Last updated: 4:37 PM Jul 14 2023

Data_Assets
Last updated: 9:40 AM Jul 20 2023

person
Last updated: 3:57 PM Jul 15 2023

New Project

Project name: Designer Exercise

Select how to start your project

- Voicebox**
Build via Voicebox
- Blank Canvas**
Build from scratch
- Import a model**
Import an existing model to edit, extend, or map to
- Connect a resource**
Leverage technical metadata from data connections to map and model

Introducing Stardog Voicebox

Start the conversation



Create Project

cloud.stardog.com/u/2/designer/#/

01

Create Project

Project Scoping

Let's get started by defining your project

Start by giving your project a title and brief description.

The collected data will help us generate better questions and data suggestions.

Title

Designer Exercise

Description

A simple use-case representing a store. Classes are customer, orders, products, categories and vendors.

Create Project



Create Project with Voicebox

←

✓ Project Scoping

01

Competency Questions

02

☰ Data Connection

03

Create Project →

What questions can Stardog answer from your data?

These will be answered by the Knowledge Graph.
Aim for 5-20 well-formed questions for the best results.

**Tips to write better questions**

- 1 Be specific and context-rich.
- 2 Focus on relationships, not just keywords.
- 3 Align questions with business/personal goals or decisions.

My questions 9 ✕

✕

Which Shippers are used for Orders sold to Customers located in the US?

✕

✕

Which Customers made Orders for Products in the Produce Category?

✕

✕

How many orders does each employee manage?

✕

✕

How many customers does each employee manage?

✕

✕

How many customers does each account manager manage?

✕

✕

How many accounts does each account manager manage?

✕

Add question +

Data Connection →

Complex data modeling and mapping

Mapping single resource – Create & Map

Selecting **Create & Map** is a great starting point for modelling and mapping a resource.

Model:

- Creates a class from the resource
- Create attributes from columns
- Creates relationships when explicitly define in the schema (Virtual Graphs)

Mapping:

- Primary Key and labels are automatically selected
- Explicit Relationships are automatically mapped (Virtual Graphs)

Publishing:

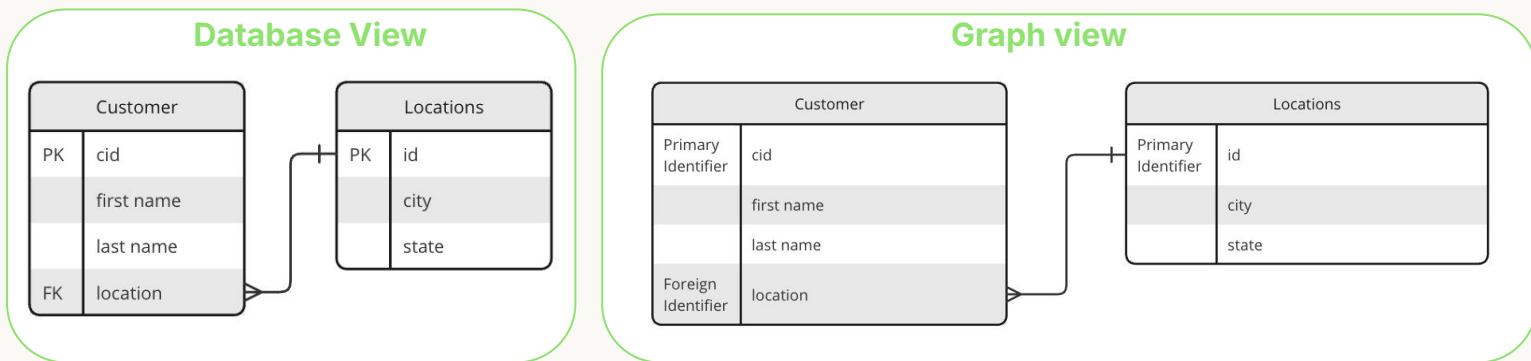
- One instance per primary identifier
- One triple per cell
- A triple for each relationship that is automatically mapped
- When primary key values are repeated, the rows are indistinguishable.
 - This is often desired behaviour when mapping flat or unnormalised files



Relationships as joins

Primary Key-Foreign Key Join <> Relationships between instances

- The foreign key in one table joins to a foreign table based on foreign key
- In RDF: relationship connects the Primary Identifier to a foreign class using the foreign identifier



The relationship can be in either direction - with with or against the join.

Tip 1: Ignore the direction of the relationship while mapping.

Tip 2: In mapping screen, Ignore (or even hide) relationships not mapped from that resource

Tip 3: Map relationships rom the resource with the foreign-key or the 'many' side of the relationship



Creating a graph from a flat file

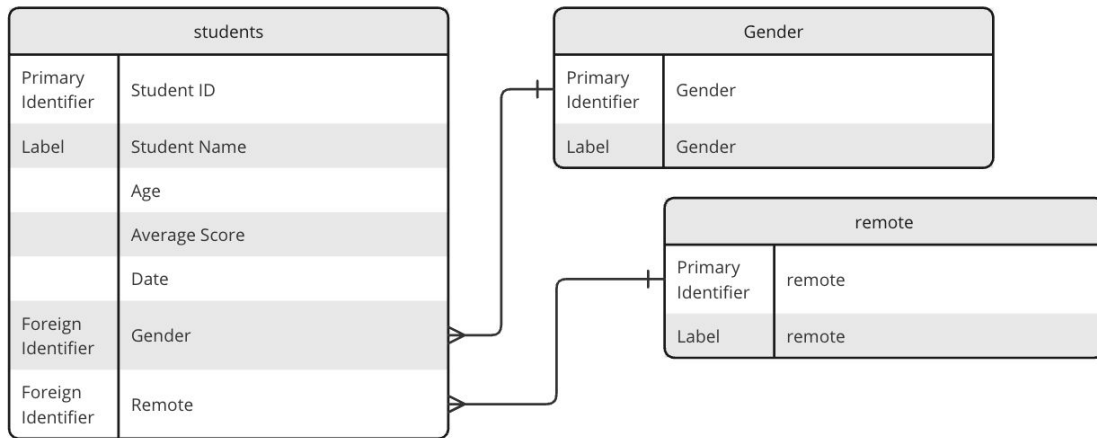
The example we saw for the students file was...:

students	
Primary Identifier	Student ID
Label	Student Name
	Age
	Average Score
	Date
	Gender
	Remote

Mapping the relationships were then the familiar type of choosing the foreign class and foreign identifier

We have no relationships to other classes

But we can create other classes, Gender and Remote and relationships by using non-unique primary identifiers

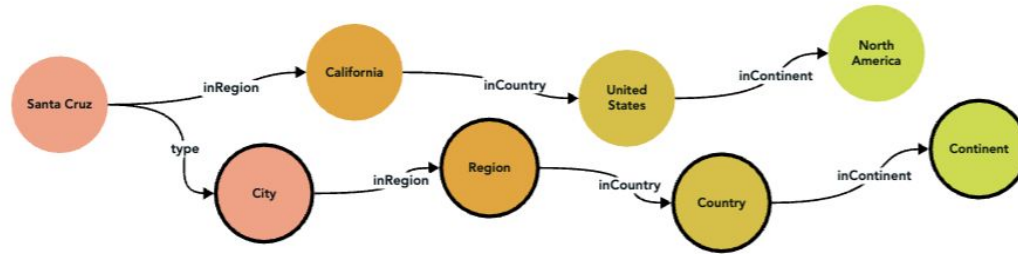


Classifications or Taxonomies

Taxonomies model hierarchical relationships between instances

- HR people classification by department, teams, geography
- Customer Segmentation
- Document Management
- Skills Management

Example: Geographies. Continents have countries, have regions, have cities



Create classes for City, Region, Country and Continent

Mapping is then familiar: Simply choose the foreign class and foreign identifier.



Many-Many Relationships

Many-to-Many Relationships are represented using Link or Junction tables

- Link tables consist of 2 (or more) primary keys to other table(s)
- Any such table can be used to map a relationship between the related tables

members.csv

Person ID	Full Name	Gender
0	Queen Elizabeth II	F
1	King Charles III	M
2	Diana, Princess of Wales	F
3	Anne, Princess Royal	F
4	Captain Mark Phillips	M
5	Andrew, Duke of York	M
6	Edward, Earl of Wessex	M
7	Sophie, Countess of Wessex	F
8	William, Prince of Wales	M
9	Catherine, Princess of Wales	F
10	Harry, Duke of Sussex	M
11	Meghan, Duchess of Sussex	F
12	Peter Phillips	M
13	Autumn Phillips	F
14	Zara Tindell	F
15	Mike Tindall	M
16	Sarah, Ducess of York	F
17	Lady Lousie Windsor	F
18	James, Viscount Severn	M
19	Princess Beatrice	F
20	Princess Eugenie	F

parent_child.csv

Parent ID	Child ID
0	1
0	3
0	5
0	6
1	8
1	10
2	8
2	10
8	27
8	28
8	29
9	27
9	28
9	29
10	25
10	26
11	25
11	26
3	12
3	14

- each member can be the child of many parents
- each parent can have many children,

the relationship we need is many-many.

This is represented in the data as a link or junction table

Each row in the link table becomes one relationship or triple in the data.

We do not need the link table as a a Class in the model (as we usually do)

Notice that a join in a relational DB becomes a first class citizen in the graph!!



Many-Many Relationships 2

Link tables are used to model many-to-many relationships.

StudentID	StudentName	Age
1	John Smith	20
2	Emily Johnson	21
3	Robert Brown	19
4	Sarah White	22
5	Michael Garcia	20
6	Linda Martin	21
7	Kevin Allen	19
8	Jessica Walker	22

CourseID	CourseName	Credits
101	Mathematics	3
102	Physics	4
103	Chemistry	3

- Each Student takes many Courses.
- Each Course has many Students.
- The link table can contain additional information about the relationship.

Here the link table also include EnrollmentDate data

We can simply model

Student →takesCourse→Course

Or we can reify the takesCourse edge by adding an Enrollment entity:

Student →hasEnrollment→ Enrollment

Enrollment →isForCourse→ Course

Enrollment →enrollmentDate →2021-09-01

Where enrollmentDate is just an attribute on an Enrollment

StudentID	CourseID	EnrollmentDate
1	C-101	2021-09-01
1	C-103	2021-09-02
2	C-102	2021-09-03
3	C-103	2021-09-04
4	C-101	2021-09-05
4	C-102	2021-09-06
5	C-101	2021-09-07
5	C-103	2021-09-08
6	C-102	2021-09-09
7	C-102	2021-09-10
7	C-103	2021-09-11
8	C-101	2021-09-12
8	C-102	2021-09-13
8	C-103	2021-09-14



Exported Files Overview



Exported Files – purpose

[Settings](#)[Export](#)[Publish](#)

To export Designer file you can explicitly [Export](#) or just [Publish](#) with the button checked



Download zipped project when publishing

You can:

- Submit these files to a Source Code Control system
- Use them to recover from a Browser crash
- Share your project with colleagues

A knowledge Graph Engineer can:

- Refine or enhance the rules you defined
- Modify the mappings to add advanced capability
- Bulk load data (using for example, CLI tools) the mappings you defined

Before Designer, these files would have been created by hand!

Designer can be thought of as a tool that takes your GUI definitions of the model, rules, virtual graphs and mappings and creates the appropriate text files as a convenience for a KG engineer.



Exported Files – summary

Name	
 course.srs	↩
 course.stardogdesigner	↩
 course.ttl	↩
▼  students	↩
 students.csv	↩
 students.properties	↩
 students.sms	↩

.srs is a Turtle file containing the Rules
.stardogdesigner contains the complete project!

- To share or import a project use this file.

- For .csv file > 2MB, only first 25 rows are stored. You will be asked to re-upload larger files at publish time.

.ttl is a Turtle serialisation of the model

Each CSV or Virtual Graph resource will have a directory with 3 files

.csv is the original csv data

.properties eg 'skip empty values'

.sms mapping file

.srs files are Stardog Rule Syntax <https://docs.stardog.com/inference-engine/user-defined-rules>

.sms files are Stardog Mapping Syntax <https://docs.stardog.com/virtual-graphs/mapping-data-sources>

.ttl files are Turtle format <https://www.w3.org/TR/turtle/>



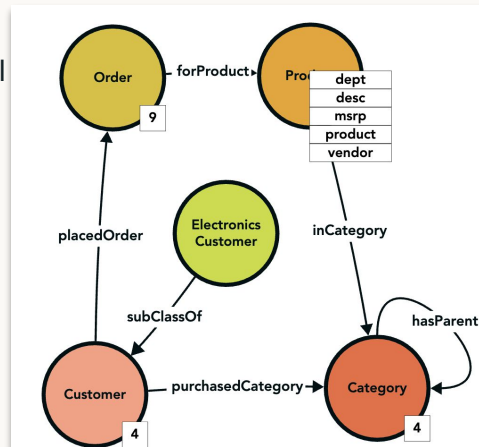
Data Model (or Schema)

A model is a description of the classes, relationships and properties of data. For Example the schema can say that Customer is a class, and that placedOrder is a relationship and that email is an attribute.

We use industry standard vocabularies to define a schema in designer. They are rdf, rdfs, schema.org and owl.

```
@prefix Designer_Exercise: <tag:stardog:designer:Designer_Exercise:model:> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

Designer_Exercise:Customer a owl:Class ;
  rdfs:comment "People who buy" ;
  rdfs:label "Customer" .
```



The data model is a lens!

In Designer, we can populate only entities defined in the model with data. For example, we might have Product Rating data but there's no way to publish the data without having schema entities.

Similarly in Explorer there is no way to explore data without relevant entities in the schema.



The exported model .ttl file

```
@prefix Designer_Exercise:
|<tag:stardog:designer:Designer_Exercise:model:> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix so: <https://schema.org/> .

Designer_Exercise:Customer a owl:Class ;
    rdfs:comment "People who buy" ;
    rdfs:label "Customer" .

Designer_Exercise:Purchases a owl:Class ;
    rdfs:label "Purchase" .

Designer_Exercise:made_purchase a owl:ObjectProperty ;
    rdfs:label "made purchase" ;
    so:domainIncludes Designer_Exercise:Customer ;
    so:rangeIncludes Designer_Exercise:Purchases .

Designer_Exercise:email a owl:DatatypeProperty ;
    rdfs:label "email" ;
    so:domainIncludes Designer_Exercise:Customer ;
    so:rangeIncludes xsd:string .
```

Creating the class called Customer:

- Defining Customer: lets explorer know to display it as a class.
- Label: lets explorer label the class

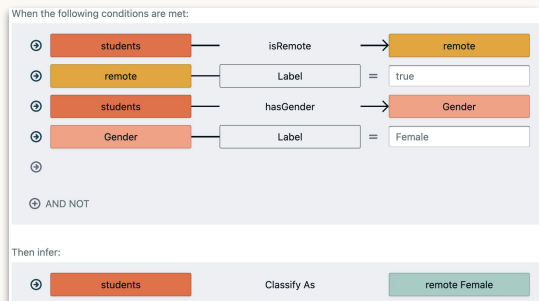
Defining the Relationships (ObjectProperty) and Attribute (DatatypeProperty)

- Explorer will populate the 'details' panel with the attributes defined in the model
- Explorer will draw the relationships you define in the model
- The domain allows Explorer query builder to limit choices in the dropdown to only valid choices
- The range lets Explorer's query builder show only the values for a string datatype (i.e. contains, equal, not equal to)

Use <https://www.easyrdf.org/converter> to apply prefixes in the .ttl to make everything more legible.



The .srs file



This is the a Stardog Rule Syntax file:

- To let the reasoner know that there is a Rule we create some triples in the model using a special namespace tag:stardog:api:rule
- The object of the triple with predicate rule:content contains the actual rule

In the example we're defining the rule for Remote Female

When the conditions in the IF portion are satisfied, the THEN portion is asserted.

In the example:

IF { the student is remote AND a female }

THEN { classify the student as Remote Female }

Stardog also supports OWL Axioms - a W3C standard for defining inference rules. OWL Axioms apart from the most basic, are difficult to write and understand.

The rules created in Designer are Stardog Rule Syntax rules that are much easier to specify and understand.

Stardog Rule Syntax <https://docs.stardog.com/inference-engine/user-defined-rules>



The exported .properties file

Add a new project resource

Source

students.csv

Resource name

students

Resource Properties

Separator character

,

Quote character

"

Escape character

"

☒ Has header

☒ Skip empty values

```
csv.header=true  
csv.quote="  
csv.separator=,  
csv.skip.empty=true
```

students.properties

There is one .properties file for each .csv resource or virtual graph

The .properties file captures your choices when adding a project resource.



The exported mapping .sms file

```
MAPPING
FROM CSV {}
TO {
  ?students_iri a model:students ;
  rdfs:label ?label ;
  model:Age ?Age_integer_field ;
  model:Date ?Date_date_field ;
  model:Gender ?Gender_string_field ;
  model:Student_ID ?Student_ID_integer_field .

  ?students_iri model:hasGender ?Gender_iri .
}
WHERE {
  BIND(TEMPLATE("tag:stardog:designer:course:data:students:{Student ID}") AS ?students_iri)
  BIND(xsd:string(sourceField("Student Name"))) AS ?label)
  BIND(StrDt(sourceField("Age"), xsd:integer) AS ?Age_integer_field)
  BIND(StrDt(sourceField("Date"), xsd:date) AS ?Date_date_field)
  BIND(StrDt(sourceField("Gender"), xsd:string) AS ?Gender_string_field)
  BIND(StrDt(sourceField("Student ID"), xsd:integer) AS ?Student_ID_integer_field)
  BIND(TEMPLATE("tag:stardog:designer:course:data:gender:{Gender}") AS ?Gender_iri)
}
```

Specifies how columns in tables or csvs are mapped to triples in rdf

Each row of the input resources is captured as triples defined in the TO section.

Each line of the TO section specifies the triples that will be created.

The WHERE section grabs the values of each cell in the row and sets the value type

[Stardog Mapping Syntax](https://docs.stardog.com/virtual-graphs/mapping-data-sources)

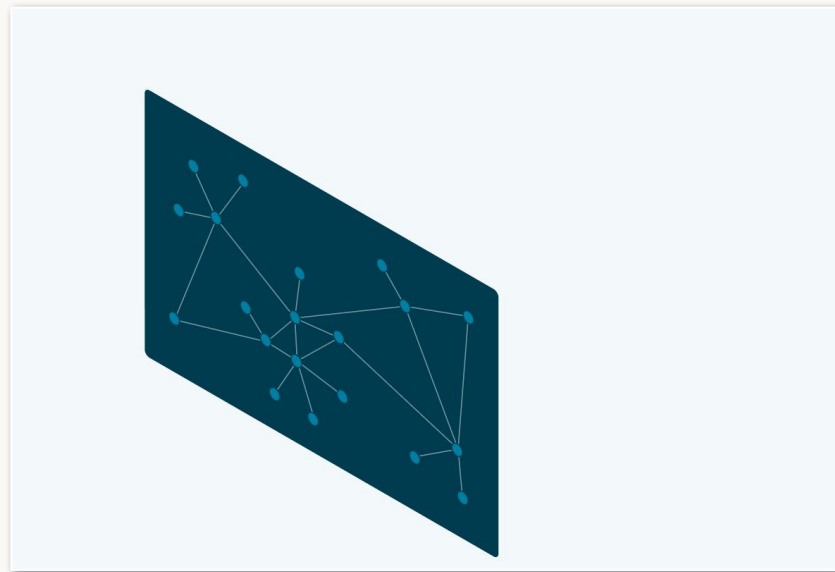
<https://docs.stardog.com/virtual-graphs/mapping-data-sources>



Virtual Graphs

Virtualization: Enables a Data fabric across all your data silos

- Connect to (almost) any data source
- Get most up to date data
 - Time based sync option
 - Cache node option
- No data copying
 - Reduces cost
 - Reduces complexity
 - Improves governance/stewardship
 - Simplifies or eliminates ETL pipelines



Result: A Data Fabric that can access any of your via the Knowledge Graph

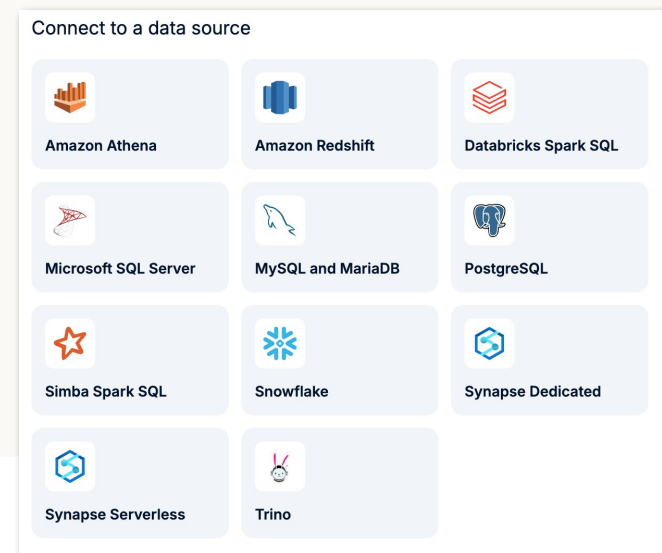
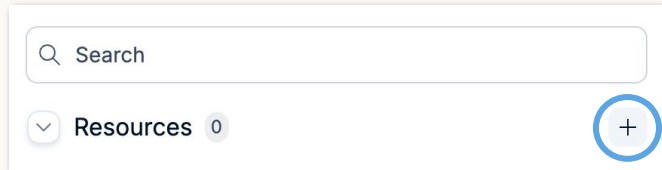


Data Sources

- Data Sources define a connection to a remote endpoint
 - Each data source can support multiple virtual graphs
 - Virtual Graphs are the mappings between the data source and the graph model
- Data Sources require a driver to be installed on the endpoint
 - Stardog Cloud environments come with pre-installed drivers
 - Licensed customers of managed endpoints can request additional drivers
 - On-prem/VPC users manage their own endpoints and install their own drivers



Data Sources in Designer



- To add a data source at any time, click the plus (+) icon next to Resources.
- When creating a new project using Voicebox, you may also access the Data Sources dialog by selecting "Data Connection."
- Virtual graphs can be created for any data source type for which a driver is installed on your Knowledge Graph endpoint.
- Choose a connection type to define a new data source or use an existing source.

Adding a new Data Source

- You can create a new data source or connect to an existing source if one exists
- Choose a connection name, and enter your credentials.
- Advanced options may be entered as key value pairs
- Choose connect. You will see a list of available tables

Connect to MySQL and MariaDB

Connection credentials

Connection name *

Username

Endpoint *

Password

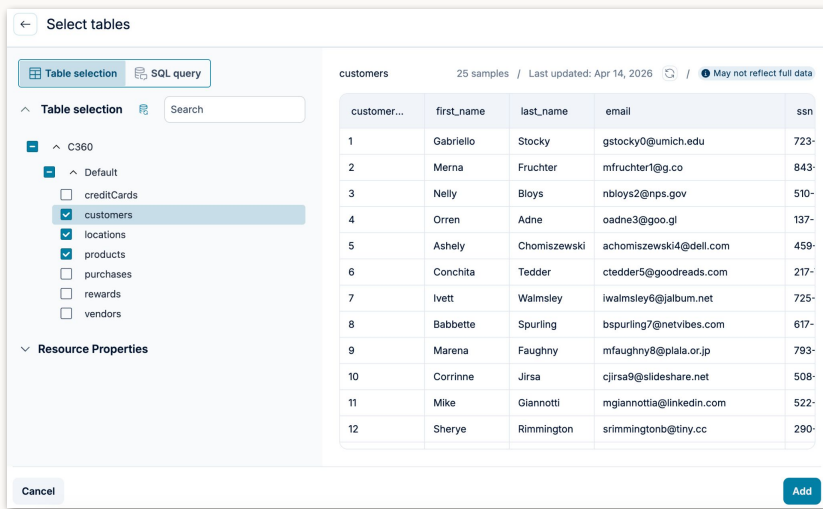
^ **Advanced Options (0)**

Property	Value
Select property	
Filter...	
abandonWhenPercentag...	
accessToUnderlyingCon...	
alternateUsernameAllowed	
commitOnReturn	
connectionProperties	
dataSource	
dataSourceJNDI	
defaultAutoCommit	

Connect



Table Selection and mapping



- Select the tables you wish to add
- Add resource properties eg SQL Function Names
- Choose Add and Create or Create and Map
- If foreign keys are defined, Create and Map will
 - create classes and attributes from tables and columns
 - automatically create relationships between classes
 - use the foreign key name and direction for those relationships

Without Voicebox, when foreign keys are not specified in the SQL schema, relationships will need to be added and mapped by hand.

With Voicebox, relationships may be created and mapped, whether or not foreign keys are specified in the SQL schema



Custom SQL query as a resource

The screenshot shows a 'Select tables' dialog box with a 'SQL query' tab selected. The query is: `SELECT c.full_name, l.city FROM 'C360':customers AS c INNER JOIN 'C360':locations AS l ON c.foreign_location = l.location_id`. Below the query is a 'Run' button. To the right, a table displays the results with columns 'full_name' and 'city'. The table contains 12 rows of data. At the bottom of the dialog are 'Cancel' and 'Add' buttons.

Select tables

Table selection SQL query

SQL Query

Run

25 samples / Last updated: Apr 14, 2026 / May not reflect full data

```
SELECT c.full_name, l.city
FROM 'C360':customers AS c
INNER JOIN 'C360':locations AS l
ON c.foreign_location = l.location_id
```

Resource Properties

SQL Quoting Convention

NATIVE

SQL Function Names

Percent Encode IRI Template Strings

Advanced Options (0)

full_name	city
Gabriello Stocky	Reading
Merna Fruchter	Dallas
Nelly Bloys	Trenton
Orren Adne	Jacksonville
Ashely Chomiszewski	Seattle
Conchita Tedder	Boise
Ivett Walmsley	Round Rock
Babbette Spurling	Honolulu
Marena Faughny	Raleigh
Corrinne Jirsa	Gaithersburg
Mike Giannotti	Arlington
Sherye Rimmington	Richmond

Cancel Add

Creates a resource that combines customer's **full_name** with their **city** by joining the customers and locations tables.

- A resource can be a table, view, or a Custom SQL query as shown
- Use Custom SQL queries
 - when the data needs to be cleaned, transformed, or converted (e.g. dates, strings, types)
 - for source-specific or proprietary join logic
- Map raw tables when you want semantic flexibility and reuse
- Keep business meaning in the ontology, and use SQL only to fix the form of the data



Publishing



Designer Projects

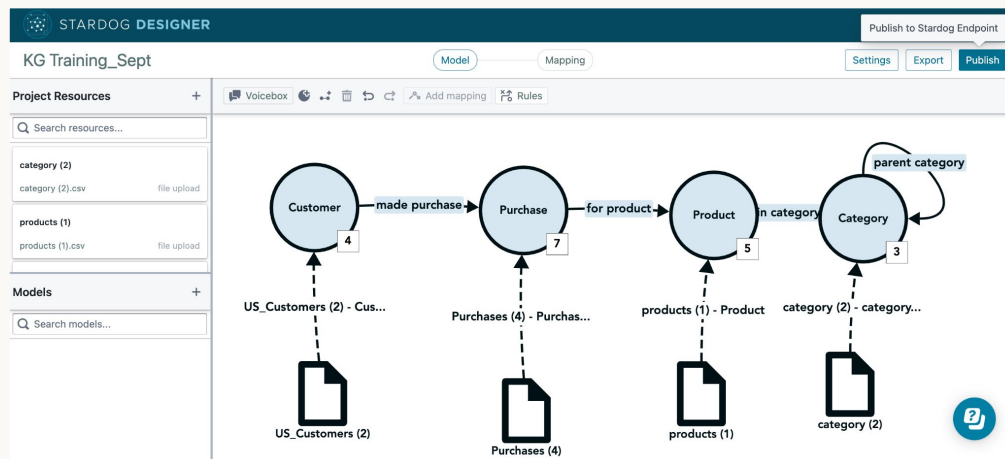
- Projects are stored by Stardog Cloud.
 - If you wish to work offline or keep project data local, the three dot menu option, Store Locally, can be used to migrate a project to use local storage.
 - Store Locally, will store your project in the browser. If you lose your browser cache you will lose your project. Make sure you export regularly.
- Export your projects!
 - For locally stored project, save them so you're not solely reliant on your browser cache.
 - You can Share the exported files
 - You should Submit them to a source code control system
- You can import a model from an existing database back into Designer
 - This imported model will have all your model entities including rules
 - Will not have your original mappings



Publishing

Publishing allows you to save your data model to a Stardog Endpoint.

During the publishing process, a user will determine what database and model to publish and configure the mapping settings.



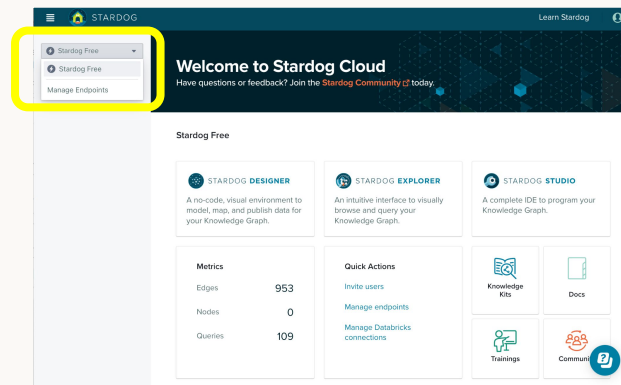
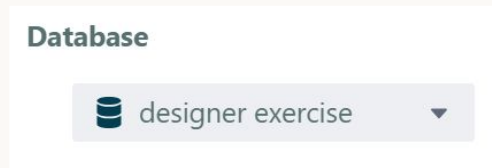
Publish Workflow

- Select the desired Stardog Endpoint

- In Designer, Click **Publish** to begin.



- Select the desired **Database**.



Add Model Namespace

- Stardog allows users to store and manage custom namespace prefix bindings for each database.
- Stored namespaces allow one to use Stardog without declaring a prefix and allow users to omit PREFIX declarations in Turtle files and SPARQL queries.
- Create a custom namespace for your data model.



Add Model Namespace ☒

Prefix	exercise
IRI 	tag:stardog:designer:Designer_Exercise:model:



Model Configuration

Users can create a new model or replace an existing model.

- For brand new projects / databases users will only be able to create a new model.
- When updating a model in Designer, users can choose whether to replace the existing model or create a second, separate model.

Designer gives users the ability to customize the data model name and the named graph the data model will be written to in the database.



Model Configuration

- ☒ Create new model
- ☐ Replace an existing model

Select a model ▼

Customize Advanced Model Details ☒

Model name Designer_Exercise

Named graph tag:stardog:designer:Designer_Exercise:model



Mapping Configuration: Files



- Users can choose to load data into a new named graph or replace / append data to an existing one.
- Designer's advanced resource details option provides the ability to customize the named graph new data will be loaded to.

customer map customer ☒ Create new ☐ Replace ☐ Append 

Customize advanced resource details ☒

Named graph urn:exercise:data:customer



Mapping Configuration: Virtual Graphs



- Users can choose to create new Virtual Graph or replace existing one.
- Designer's advanced resource details option provides the ability to customize the name of the Virtual Graph.

customer map customer vg ☒ Create new ☐ Replace 

Customize advanced resource details ☒

Virtual graph designer_exercise_vg



Publish Project

- Users can publish a project after they have:
 - Selected an Endpoint and Database
 - Configured the Model
 - Configured the Mapping(s)
- Before publishing users should select whether or not they would like to download a zipped version of the project when publishing.
 - The zipped file will contain all model, source data files/source configurations and mapping files required to build and deploy the project.



Download zipped project when publishing

Back

Publish



- Once satisfied with all project configurations, users can **Publish** the project to the Stardog server.

Sensitive Properties



Fine-grained Security

- Define sensitive property groups
- Associate attributes to sensitive property groups
- Sensitive property groups can be managed via the AWS IAM Identity Center console, AWS IAM Identity Center Studio or the CLI

Create Sensitive Property Group

Name

New Sensitive Property Group

Attributes

Attribute Name	Data Type	Rule
		Add
ssn	string	Create

Sensitive Property Groups

	Name	Other attributes
☒	PII	email, gender, and 2 more



Infer class types based on Shared Properties

If an individual or instance has certain properties that are characteristic of a particular class, a reasoner can deduce that the individual belongs to that class, even if it's not explicitly stated.

Project Settings

×

Project Name

Designer Exercise

Customize advanced project details

☒

Model Namespace

tag:stardog:designer:Designer_Exercise:model:

Instance Data Namespace

tag:stardog:designer:Designer_Exercise:data:

Infer class types based on shared properties ⓘ

☐ Yes ☒ No ☐ Custom per property

This behavior is sometimes desirable and can be enabled by checking Yes in the project settings and it may be customized by property.

For example, I might want to say that Customer has a Name. Thereafter I want everything that has a Name to automatically be made an instance of Customer.

In most cases this is not what we want. Usually we would like Customer or Product to have a Name, without inferring that a Product is a Customer.

For this reason the default behavior is to not infer class types.



Data Model Multi-tenancy



Why Multiple Data Models?

Enable Explorer users to use different data models on the same underlying dataset:

Separate Domains

Maintain distinct data models for different business units such as Sales and HR, each with domain-specific vocabulary and structures

Multiple Use Cases

Supports diverse interpretation needs without reshaping the data

Reusability

Aligns with core philosophy in data mesh or reusable data product architecture

Ontology Extension

Import established ontologies from external sources and extend (or reduce) them with your organisation's unique requirements



Typical Project Arrangement

Base Data Model

Purpose: The trusted, mapped representation of the data.

Includes: Base data model and mapping to source systems

Example Structure:

Customer → Purchase → Product → Category

Characteristics:

- Stable/Safe 'default' model
- Close to source data
- No viewpoint-specific simplification
- Published to a database

Base Project

Mapping stays stable and governed
Lenses can evolve independently
Each business domain gets its own "view"
Reasoning can add perspective-specific entities
Clean separation of data engineering and semantic modeling

Domain Specific Data Models

Purpose: Alternative lens over the same data.

Includes: Modified/reduced/enhanced Data model.
Optional derived relationships (via reasoning). No mappings

Example Alternative Lens:

Category → orderedByCustomer → Customer

- Business-specific
- Simplified
- Evolvable without breaking mappings
- Multiple lenses possible

Multi-model Project



Typical Project Arrangement Advantages

- Mapping stays stable and governed
- Domain specific Lenses can evolve independently
- Each business domain gets its own “view”
- Reasoning adds perspective-specific entities
- Clean separation of data engineering and semantic modeling





All

Search your knowledge graph



Query Builder



Search

.NET Misconfi...

/etc/passwd an...

Absolute Path ...

Abuse Elevatio...

Academic Pap...

Acceptance of ...

Access

Access Contro...

Access Contro...

Access Contro...

Access Contro...

Access Contro...

Access Contro...

Access Denied...

Access Grante...

Access Mediat...

Access Mediat...

Access Mediator

Access Modeling

Access of Me...

Access of Me...

Access of Res...

Access of Unin...

Access Policy ...

Voicebox

Imported Ontologies are great!!

But often bloated and cumbersome - Help!

Practical requirements

For very large Ontologies Designer/Explorer may not be the most practical tool:

However

- In most projects, your instance data uses a fraction of the full Ontology
- Most use cases need only a focused subset that is much more Designer/Explorer-friendly.
- You can reduce a large ontology manually, but it's usually better to have a Knowledge Graph engineer generate "reduced" models for specific use cases using a SPARQL Update language.



Making Data Model Relevant to a User

The problem:

In connecting more and more fragmented data sources (eg in end-to-end use cases) the data model necessarily scales too.

- Large models become difficult to manage and visualise in Designer
- Explorer shows users lots of things they may not care about.

The Solution:

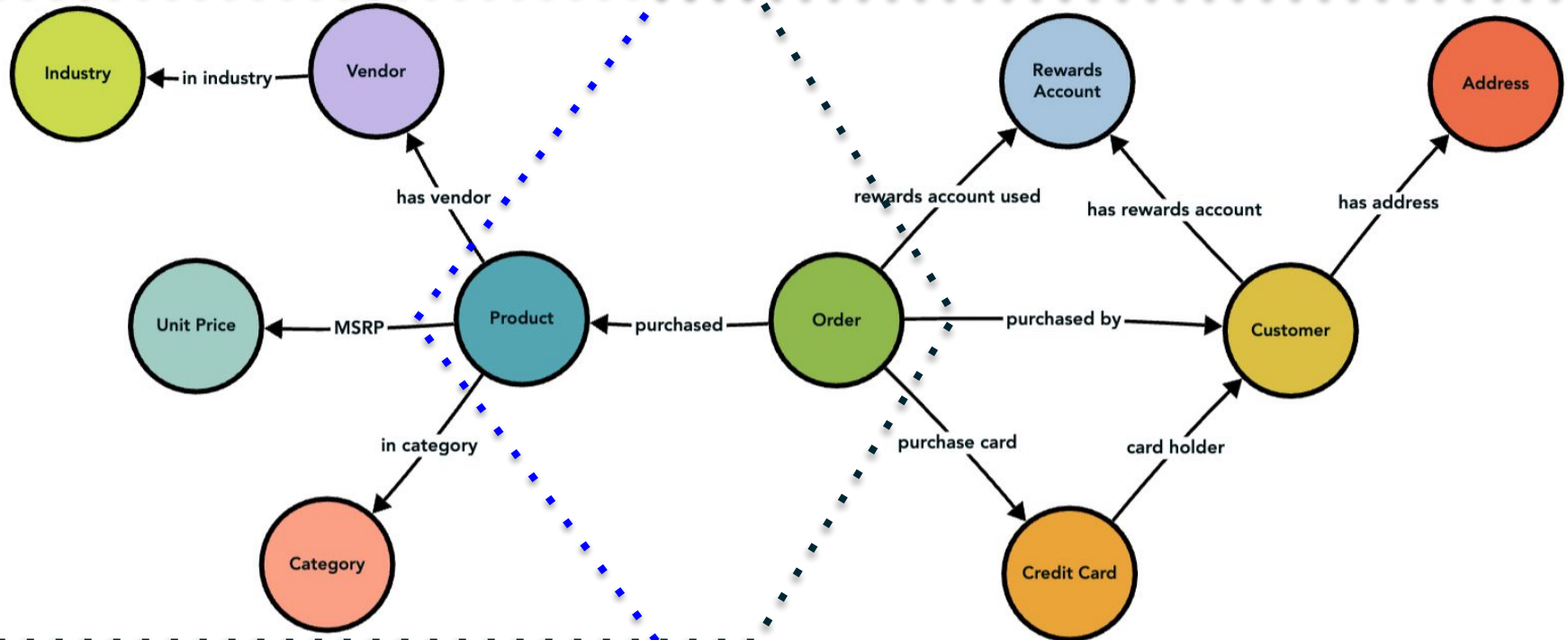
- Break the model into smaller, domain-focused slices.

The Benefit:

- Explorer shows only what matters
 - Exploration becomes faster and less intimidating
- Multiple Designer users can work simultaneously
 - on their own domain
 - yet still contribute to the same Main (full) model



The Main Data Model - C360 Knowledge Kit

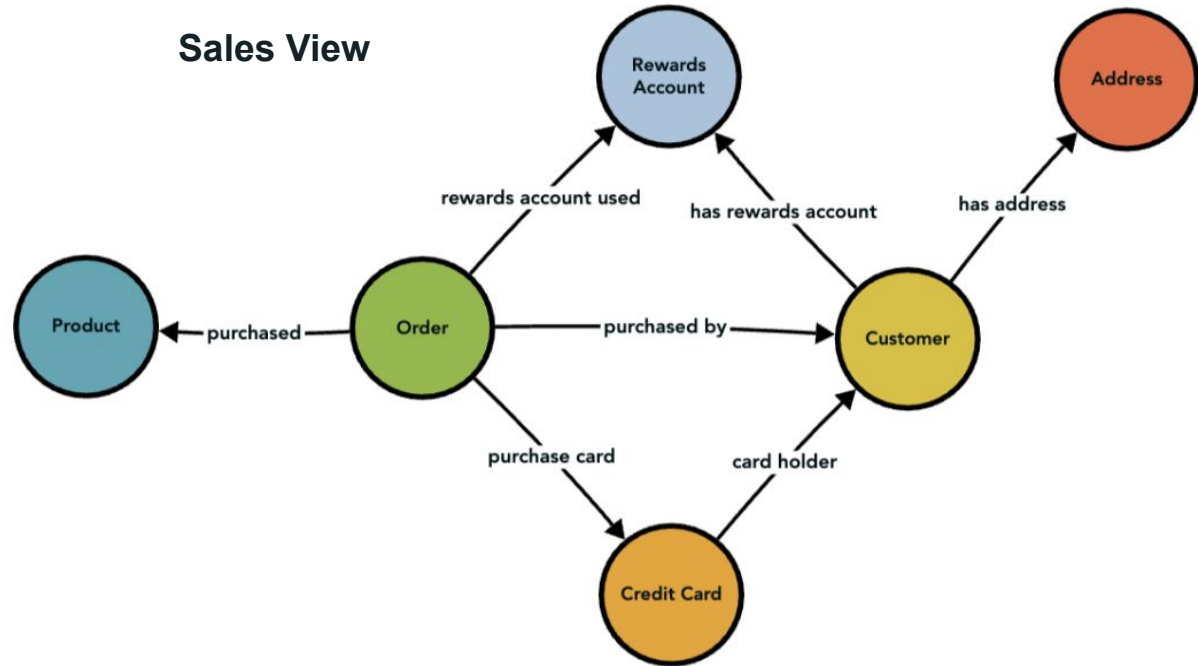


The Sales Data Model

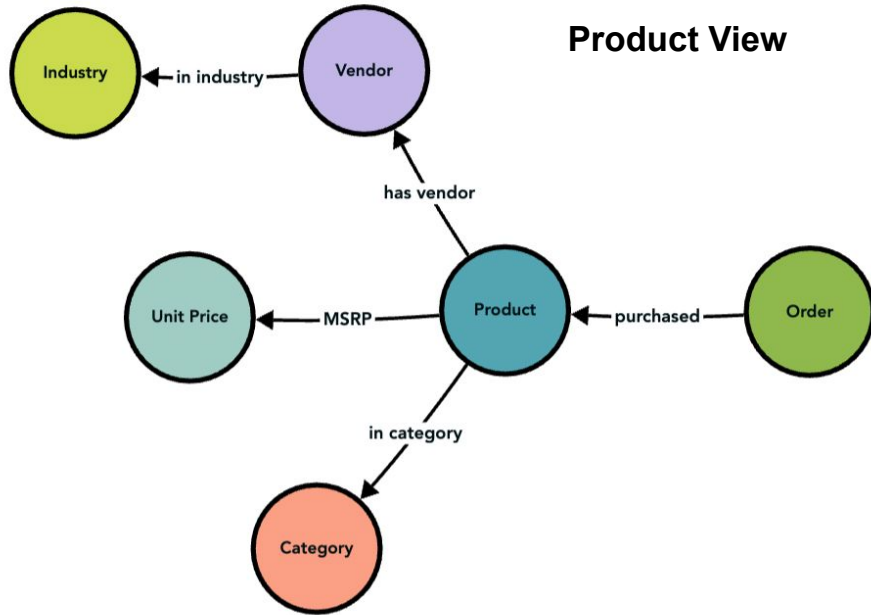
Sales doesn't care about Product and Shipping detail. They care about Customers.

Same data, different perspectives.

- Example: **"High-value product"** doesn't mean premium specs. It means a product that consistently drives high-value orders.



The Product Data Model



Product Management has no interest in credit card or rewards accounts

They look at the same data through a different lens.

Example: A **valued product** is one which is sold in higher quantities per order

This means

- single picking point
- single packing process
- single delivery address

Sales will naturally define “valuable” differently



Review

Review

1 Understanding Triples, IRIs & Namespaces

2 Complex data modeling and mapping

3 Map using Virtual Graphs

4 Sensitive Properties

5 Read exported files

6 Publishing production ready data models



Questions?



Thank You!

To contact Stardog about training please email:
training@stardog.com



